



Experiment No. 6

Title: Mutual Exclusion Algorithm

Aim: Program to demonstrate Mutual Exclusion Algorithm using Python.

Theory: Mutual exclusion is a critical concept in concurrency control that helps prevent race conditions in a multi-process or multi-threaded environment. It ensures that only one process is allowed to execute the critical section at any given time, while all other concurrent processes are blocked until the first process exits the critical section.

In this experiment, we will explore mutual exclusion algorithms and compare their performance in single computer systems versus distributed systems. Mutual exclusion is a fundamental concept in concurrent programming that ensures that only one process or thread is allowed to execute a critical section at a time. It is necessary to prevent race conditions where multiple processes access the same resource concurrently, leading to unpredictable results or errors.

Mutual exclusion can be implemented using various algorithms, including software-based and hardware-based approaches. In a single computer system, memory and other resources are shared between different processes, making it easier to implement mutual exclusion. Shared variables such as semaphores or locks can be used to control access to the critical section.

A semaphore is a variable that can be used to control access to shared resources in a concurrent system. It can be implemented as a simple binary semaphore that signals whether the resource is available or not. When a process wants to access the resource, it waits on the semaphore until it is signaled, indicating that the resource is available. In distributed systems, mutual exclusion is more challenging to implement because there is no shared memory or common physical clock. Therefore, message passing is used to coordinate access to shared resources.

The messages exchanged between processes can be used to request permission to access the critical section or release the resource after use. One commonly used algorithm in distributed systems is the Lamport's bakery algorithm, which assigns a ticket number to each process and allows only the process with the smallest ticket number to enter the critical section. In conclusion, mutual exclusion is an essential concept in concurrent programming that ensures the correct behaviour of shared resources in a multi-process or multi-threaded environment. It can be implemented using different algorithms depending on the system's characteristics and requirements.

In a single computer system, shared variables such as semaphores can be used to implement mutual exclusion, while in a distributed system, message passing algorithms such as the Lamport's bakery algorithm can be used.

Requirements of Mutual exclusion Algorithm:

- 1. No Deadlock:** Two or more sites should not endlessly wait for any message that will never arrive.
- 2. No Starvation:** Every site who wants to execute a critical section should get an opportunity to execute it in finite time. Any site should not wait indefinitely to execute a critical section while other sites are repeatedly executing a critical section.
- 3. Fairness:** Each site should get a fair chance to execute a critical section. Any request to execute a critical section must be executed in the order they are made i.e Critical section execution requests should be executed in the order of their arrival in the system.
- 4. Fault Tolerance:**
In case of failure, it should be able to recognize it by itself in order to continue functioning without any disruption.

Solution to distributed mutual exclusion:

As we know shared variables or a local kernel cannot be used to implement mutual exclusion in distributed systems. Message passing is a way to implement mutual exclusion. Below are the three approaches based on message passing to implement mutual exclusion in distributed systems:

1. Token Based Algorithm:

- A unique token is shared among all the sites.
- If a site possesses the unique token, it is allowed to enter its critical section
- This approach uses sequence numbers to order requests for the critical section.
- Each requests for the critical section contains a sequence number.
- This sequence number is used to distinguish old and current requests.
- This approach insures Mutual exclusion as the token is unique
- Example: **Suzuki-Kasami's Broadcast Algorithm**

2. Non-token based approach:

- A site communicates with other sites in order to determine which sites should execute the critical section next. This requires the exchange of two or more successive rounds of messages among sites.
- This approach uses timestamps instead of sequence number to order requests for the critical section.
- Whenever a site makes a request for a critical section, it gets a timestamp. Timestamp is also used to resolve any conflict between critical section requests.
- Example: **Lamport's algorithm, Ricart-Agrawala algorithm**

3. Quorum based approach:

- Instead of requesting permission to execute the critical section from all other sites, Each site requests only a subset of sites which is called a quorum.
- Any two subsets of sites or Quorum contains a common site.
- This common site is responsible to ensure mutual exclusion
- Example: **Maekawa's Algorithm**

PROGRAM:

```
from collections import deque
# Process IDs
p1 = 1
p2 = 2
p3 = 3

# flag = 0 means critical section is free
# flag = 1 means critical section is occupied
flag = 0
cs = 0 # current process in critical section
q = deque() # queue of waiting processes

def request_cs(process):
```



```
global flag, cs
if flag == 0:
    cs = process
    flag = 1
    print("System Status:")
    print(f"Critical section is occupied by: p{cs}")
else:
    print(f"Process p{cs} is already in critical section.")
    q.append(process)
    print("System Status:")
    print(f"Critical section is occupied by: p{cs}")
    print(f"Processes waiting are: {list(q)}")

def release_cs():
    global flag, cs
    print(f"The process p{cs} is removed from the critical section.")
    if q:
        cs = q[0]
        q.popleft()
        print("System Status:")
        print(f"Critical section occupied by p{cs}")
    else:
        print("No process is waiting in the queue")
        flag = 0

def main():
    while True:
        print("\n....menu....")
        print("1. Request the critical section")
        print("2. Release the critical section")
        print("3. Exit")
        opt0 = int(input("Enter choice: "))

        if opt0 == 1:
            print("Select the process.")
            print("1. p1")
            print("2. p2")
            print("3. p3")
            opt1 = int(input("Enter choice: "))
            if opt1 == 1:
                request_cs(p1)
            elif opt1 == 2:
                request_cs(p2)
            elif opt1 == 3:
                request_cs(p3)
            else:
                print("Invalid process selection.")

        elif opt0 == 2:
            release_cs()

        elif opt0 == 3:
            print("Exiting...")
            break
```



else:

print("Invalid choice.")

if __name__ == "__main__":

main()

OUTPUT:

```
C:\Users\DELL\Desktop\dc 1-5>python dc6.py
```

```
....menu....
1. Request the critical section
2. Release the critical section
3. Exit
Enter choice: 1
Select the process.
1. p1
2. p2
3. p3
Enter choice: 1
System Status:
Critical section is occupied by: p1
```

```
....menu....
1. Request the critical section
2. Release the critical section
3. Exit
Enter choice: 1
Select the process.
1. p1
2. p2
3. p3
Enter choice: 2
Process p1 is already in critical section.
System Status:
Critical section is occupied by: p1
Processes waiting are: [2]
```

```
....menu....
1. Request the critical section
2. Release the critical section
3. Exit
Enter choice: 1
Select the process.
1. p1
2. p2
3. p3
Enter choice: 3
Process p1 is already in critical section.
System Status:
Critical section is occupied by: p1
Processes waiting are: [2, 3]
```

```
....menu....
1. Request the critical section
2. Release the critical section
3. Exit
Enter choice: 2
The process p1 is removed from the critical section.
System Status:
Critical section occupied by p2
```

```
....menu....
1. Request the critical section
2. Release the critical section
3. Exit
Enter choice: 2
The process p2 is removed from the critical section.
System Status:
Critical section occupied by p3
```

```
....menu....
1. Request the critical section
2. Release the critical section
3. Exit
Enter choice: 2
The process p3 is removed from the critical section.
No process is waiting in the queue
```

```
....menu....
1. Request the critical section
2. Release the critical section
3. Exit
Enter choice: 3
Exiting...
```

```
C:\Users\DELL\Desktop\dc 1-5>
```

● Update is ready, click to restart.

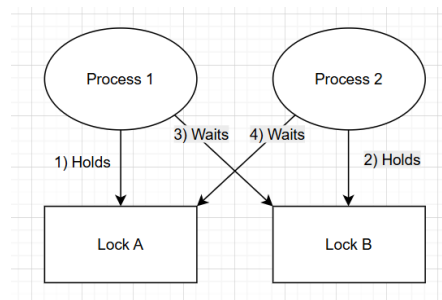


Experiment No. 7

Title: Deadlock Management in Distributed System

Aim: Program to demonstrate Deadlock Management in Distributed Systems.

Theory: Deadlocks can occur in distributed computing systems when multiple processes or nodes compete for shared resources, leading to a situation where none of the processes can proceed further. Deadlocks can result in significant performance degradation, resource wastage, and system failure.



Deadlock management is an important concern in distributed computing as it can lead to performance degradation, resource wastage, and system failure. Deadlocks occur when multiple processes compete for shared resources and get stuck waiting for resources that are currently held by other processes. In distributed computing, deadlocks can be even more complex and challenging to manage as processes may be located across different machines or networks. There are several techniques for managing deadlocks in distributed computing, including resource allocation and release, deadlock detection, deadlock prevention, deadlock avoidance, and recovery. One commonly used technique for managing deadlocks is mutual exclusion. Mutual exclusion is a mechanism that ensures that only one process can access a shared resource at any given time. In distributed computing, mutual exclusion can be implemented using locks, semaphores, or other synchronization primitives. When a process requests access to a shared resource, it must acquire the corresponding lock or semaphore. If the lock or semaphore is currently held by another process, the requesting process will be blocked until the lock or semaphore becomes available. By using mutual exclusion, deadlocks can be prevented by ensuring that processes are granted access to shared resources in a controlled and coordinated manner. For example, if two processes need access to two shared resources, they can be granted access to the resources in a specific order to prevent a potential deadlock. If a process is unable to acquire a lock or semaphore, it can be designed to release any locks it currently holds to avoid a potential deadlock.

However, it is important to note that mutual exclusion can also lead to potential deadlocks if not implemented correctly. For example, if two processes hold different locks and attempt to acquire each other's lock, they can get stuck waiting indefinitely for the other process to release the lock, resulting in a deadlock. To effectively manage deadlocks in distributed computing using mutual exclusion, it is essential to carefully design and implement the locking mechanism, ensuring that locks are acquired and released in a controlled and coordinated manner. Deadlock prevention, detection, avoidance, and recovery techniques can also be used in combination with mutual exclusion to provide a comprehensive deadlock management strategy.

PROGRAM:

```
import threading
```

```
import time
import random
# Two locks (equivalent to Java's ReentrantLock)
lock1 = threading.Lock()
lock2 = threading.Lock()

def acquire_locks(first_lock, second_lock, thread_name):
    """
    Try to acquire both locks.
    If second lock unavailable, release first lock and retry.
    This prevents deadlock.
    """
    while True:
        first_lock_acquired = False
        second_lock_acquired = False

        try:
            # Try to acquire first lock (non-blocking)
            first_lock_acquired = first_lock.acquire(blocking=False)

            if first_lock_acquired:
                # Try to acquire second lock (non-blocking)
                second_lock_acquired = second_lock.acquire(blocking=False)

                if second_lock_acquired:
                    # Both locks acquired - enter critical section
                    print(f"{thread_name} acquired both locks,"
                          f" performing critical section.")
                    break # Exit the loop

        finally:
            # If second lock not acquired, release first lock and retry
            if not second_lock_acquired:
                if first_lock_acquired:
                    first_lock.release()
                    print(f"{thread_name} could not acquire both locks,"
                          f" retrying...")
                    # Small random sleep to avoid livelock
                    time.sleep(random.uniform(0.01, 0.1))
```



```
# Release both locks after critical section
second_lock.release()
first_lock.release()
print(f"{thread_name} released both locks.")

def main():
    # Thread 1 acquires locks in order: lock1 -> lock2
    t1 = threading.Thread(
        target=acquire_locks,
        args=(lock1, lock2, "Thread-1"),
        name="Thread-1"
    )
    # Thread 2 acquires locks in opposite order: lock2 -> lock1
    # This would cause deadlock without tryLock mechanism
    t2 = threading.Thread(
        target=acquire_locks,
        args=(lock2, lock1, "Thread-2"),
        name="Thread-2"
    )
    # Start both threads
    t1.start()
    t2.start()
    # Wait for both threads to finish
    t1.join()
    t2.join()
    print("\nBoth threads completed successfully. No deadlock occurred.")

if __name__ == "__main__":
    main()
```

Output:

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECK
Microsoft Windows [Version 10.0.26200.7705]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DELL\Desktop\dc 1-5>python dc7.py
Thread-1 acquired both locks, performing critical section.
Thread-1 released both locks.
Thread-2 acquired both locks, performing critical section.
Thread-2 released both locks.

Both threads completed successfully. No deadlock occurred.

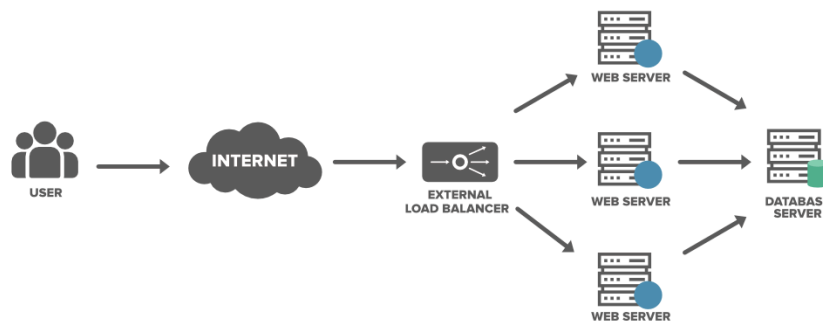
C:\Users\DELL\Desktop\dc 1-5>
```

Experiment No. 8

Title: Load-balancing approach in a distributed environment

Aim: To implement the program for demonstrating a load-balancing approach in a distributed environment.

Theory: A load balancer is a device that acts as a reverse proxy and distributes network or application traffic across a number of servers. Load adjusting is the approach to conveying load units (i.e., occupations/assignments) across the organization which is associated with the distributed system. Load adjusting should be possible by the load balancer. The load balancer is a framework that can deal with the load and is utilized to disperse the assignments to the servers. The load balancers allocate the primary undertaking to the main server and the second assignment to the second server.



Load balancing model in Distributed System.

Purpose of Load Balancing in Distributed Systems:

Security: A load balancer provides safety to your site with practically no progressions to your application. Protect applications from emerging threats: The Web Application Firewall (WAF) in the load balancer shields your site.

Authenticate User Access: The load balancer can demand a username and secret key prior to conceding admittance to your site to safeguard against unapproved access.

Protect against DDoS attacks: The load balancer can distinguish and drop conveyed refusal of administration (DDoS) traffic before it gets to your site.

Performance: Load balancers can decrease the load on your web servers and advance traffic for a superior client experience.

SSL Offload: Protecting traffic with SSL (Secure Sockets Layer) on the load balancer eliminates the upward from web servers bringing about additional assets being accessible for your web application.

Traffic Compression: A load balancer can pack site traffic giving your clients a vastly improved encounter with your site.

Load Balancing Approaches:

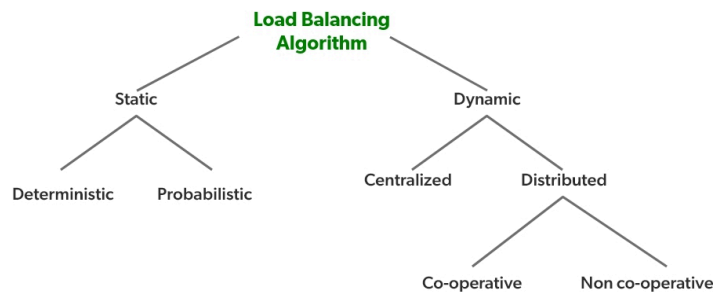
- Round Robin
- Least Connections
- Least Time
- Hash
- IP Hash

Classes of Load Adjusting Calculations:

Following are a portion of the various classes of the load adjusting calculations.

- **Static:** In this model assuming any hub/node is found with a heavy load, an assignment can be taken arbitrarily and move the undertaking to some other arbitrary system.
- **Dynamic:** It involves the present status data for load adjusting. These are better calculations than static calculations.
- **Deterministic:** These calculations utilize processor and cycle attributes to apportion cycles to the hubs.
- **Centralized:** The framework states data is gathered by a single hub.

Load Balancing Algorithm:



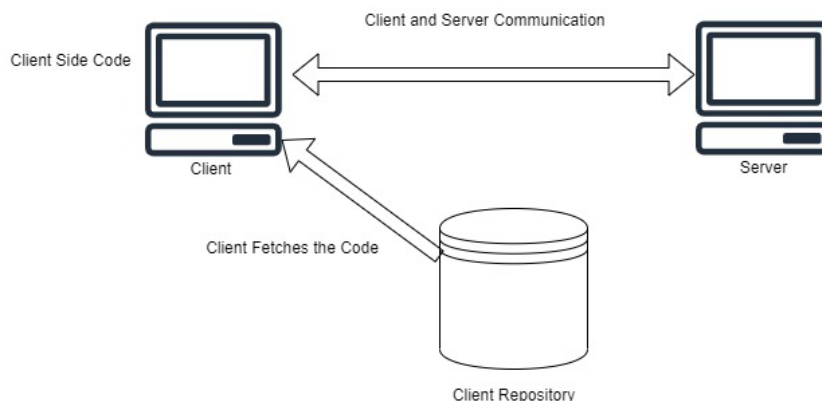
Advantages of Load Balancing:

- Load balancers minimize server response time and maximize throughput.
 - Load balancer ensures high availability and reliability by sending requests only to online servers.
- Load balancers do continuous health checks to monitor the server's capability of handling the request.

Migration:

Another important policy to be used by a distributed operating system that supports process migration is to decide about the total number of times a process should be allowed to migrate.

Migration Process in Distributed System:



Migration Models:

- Code section
- Resource section
- Execution section
- Migration Model

Code section: It contains the real code.

Resource fragment: It contains a reference to outer resources required by the interaction. **Execution**

section: It stores the ongoing execution condition of interaction, comprising private information, the stack, and the program counter.

Powerless movement: In the powerless relocation just the code section will be moved. **Solid relocation:** In this movement, both the code fragment and the execution portion will be moved. The relocation additionally can be started by the source.

Code:

```
def print_load(servers, processes):
    each = processes // servers # base processes per server
    extra = processes % servers # remaining processes to distribute

    print("\n--- Current Load Distribution ---")
    for i in range(extra):
        print(f"Server {i + 1} has {each + 1} Processes")

    for i in range(extra, servers):
        print(f"Server {i + 1} has {each} Processes")
    print("-----")

def main():
    servers = int(input("Enter the number of Servers: "))
    processes = int(input("Enter the number of Processes: "))

    while True:
        print_load(servers, processes)

        print("\n1. Add Servers")
        print("2. Remove Servers")
        print("3. Add Processes")
        print("4. Remove Processes")
        print("5. Exit")

        choice = int(input("Enter choice: "))

        if choice == 1:
            n = int(input("How many more servers to add? "))
            servers += n
            print(f"Servers increased to {servers}.")
```

```
elif choice == 2:
    n = int(input("How many servers to remove? "))
    if servers - n <= 0:
        print("Cannot remove. At least 1 server must remain.")
    else:
        servers -= n
        print(f"Servers decreased to {servers}.")
```

```
elif choice == 3:
    n = int(input("How many more Processes to add? "))
    processes += n
    print(f"Processes increased to {processes}.")
```

```
elif choice == 4:
    n = int(input("How many Processes to remove? "))
    if processes - n < 0:
        print("Cannot remove. Processes cannot be negative.")
    else:
        processes -= n
        print(f"Processes decreased to {processes}.")
```

```
elif choice == 5:
    print("Exiting...")
    break
```

```
else:
    print("Invalid choice. Please try again.")
```

```
if __name__ == "__main__":
    main()
```

OUTPUT:

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS S
C:\Users\DELL\Desktop\dc 1-5>python dc8.py
Enter the number of Servers: 3
Enter the number of Processes: 10

--- Current Load Distribution ---
Server 1 has 4 Processes
Server 2 has 3 Processes
Server 3 has 3 Processes
-----
```



SHREE L. R. TIWARI COLLEGE OF ENGINEERING

An Autonomous Institute Affiliated to University of Mumbai,
Approved by AICTE & DTE, Maharashtra State | NAAC Accredited, NBA Accredited Program | ISO 9001:2015 Certified |
DTE Code No: 3423 | Recognized under Section 2(f) of the UGC Act 1956 | Minority Status (Hindi Linguistic)

```
1. Add Servers
2. Remove Servers
3. Add Processes
4. Remove Processes
5. Exit
Enter choice: 1
How many more servers to add? 2
Servers increased to 5.
```

```
--- Current Load Distribution ---
Server 1 has 2 Processes
Server 2 has 2 Processes
Server 3 has 2 Processes
Server 4 has 2 Processes
Server 5 has 2 Processes
-----
```

```
1. Add Servers
2. Remove Servers
3. Add Processes
4. Remove Processes
5. Exit
Enter choice: 3
How many more Processes to add? 3
Processes increased to 13.
```

```
--- Current Load Distribution ---
Server 1 has 3 Processes
Server 2 has 3 Processes
Server 3 has 3 Processes
Server 4 has 2 Processes
Server 5 has 2 Processes
-----
```

```
1. Add Servers
2. Remove Servers
3. Add Processes
4. Remove Processes
5. Exit
Enter choice: 2
How many servers to remove? 2
Servers decreased to 3.
```

```
--- Current Load Distribution ---
Server 1 has 5 Processes
Server 2 has 4 Processes
Server 3 has 4 Processes
-----
```

```
1. Add Servers
2. Remove Servers
3. Add Processes
4. Remove Processes
5. Exit
Enter choice: 4
How many Processes to remove? 4
Processes decreased to 9.
```

```
--- Current Load Distribution ---
Server 1 has 3 Processes
Server 2 has 3 Processes
Server 3 has 3 Processes
-----
```

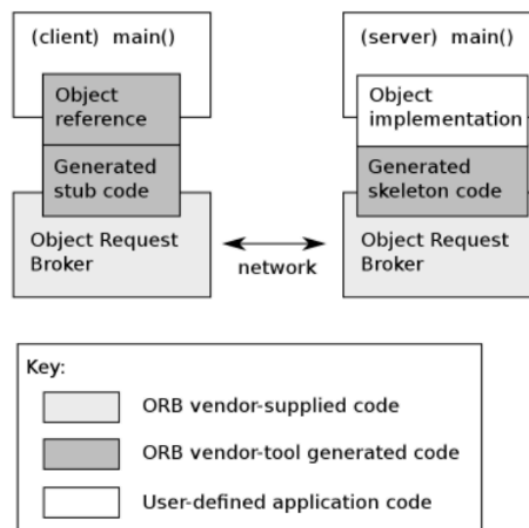
```
1. Add Servers
2. Remove Servers
3. Add Processes
4. Remove Processes
5. Exit
Enter choice: 5
Exiting...
```

```
C:\Users\DELL\Desktop\dc 1-5>
```

Experiment No. 9

Aim: Case study of CORBA.

OVERVIEW: BEA WebLogic Enterprise (WLE) is an implementation of the Java 2 Enterprise Edition (J2EE) platform. As such, WLE includes CORBA (Common Object Request Broker Architecture) capability for standards-based interoperability and connectivity. The WLE platform allows distributed Web-enabled Java applications to transparently invoke operations on remote network services using the industry standard Object Management Group (OMG) Interface Definition Language (IDL) and Internet Inter-ORBProtocol (IIOP) defined by the OMG. Runtime components include a fully compliant Java Object Request Broker (ORB) for distributed computing using IIOP communication. To build Java applications that can access CORBA objects, you need the BEA idltojava compiler, a tool that converts IDL files to Java stub and skeleton files. The idltojava compiler is included with the BEA WebLogic Enterprise software.



Steps needed to write the IDL for the Hello World application.

Step 1: Declaring the CORBA IDL Module

A CORBA module is a namespace that acts as a container for related interfaces and declarations. It corresponds closely to a Java package. Each module statement in an IDL file is mapped to a Java package statement.

```
module HelloApp {  
    // Add subsequent lines of code here.  
};
```

Step 2: Declaring the Interface

Like Java interfaces, CORBA interfaces declare the API contract that an object has with other objects. Each interface statement in the IDL maps to a Java interface statement when mapped.

In your Hello.idl file, enter the interface statement:

```
module HelloApp {  
    interface Hello // Add
```



```
{ // these  
// four  
}; // lines.  
};
```

The following describes some of the most significant ways that CORBA can be used to facilitate communication among distributed objects.

Objects by Reference: This reference is either acquired through a stringified Uniform Resource Locator (URL), NameService lookup (similar to Domain Name System (DNS)), or passed-in as a method parameter during a call. Object references are lightweight objects matching the interface of the real object (remote or local). Method calls on the reference result in subsequent calls to the ORB and blocking on the thread while waiting for a reply, success or failure. The parameters, return data (if any), and exception data are marshaled internally by the ORB according to the local language and OS mapping.

Data By Value: The CORBA Interface Definition Language provides the language- and OS-neutral inter object communication definition. CORBA Objects are passed by reference, while data (integers, doubles, structs, enums, etc.) are passed by value. The combination of Objects-by-reference and data-by-value provides the means to enforce great data typing while compiling clients and servers, yet preserve the flexibility inherent in the CORBA problem-space.

Objects By Value (OBV): Apart from remote objects, the CORBA and RMI-IIOP define the concept of the OBV and Value Types. The code inside the methods of Value Type objects is executed locally by default. If the OBV has been received from the remote side, the needed code must be either a priori known for both sides or dynamically downloaded from the sender. To make this possible, the record, defining OBV, contains the Code Base that is a space-separated list of URLs whence this code should be downloaded. The OBV can also have remote methods.

CORBA Component Model (CCM)

CORBA Component Model (CCM) is an addition to the family of CORBA definitions. It was introduced with CORBA 3 and it describes a standard application framework for CORBA components. Though not dependent on "language dependent Enterprise Java Beans (EJB)", it is a more general form of EJB, providing four component types instead of the two that EJB defines. It provides an abstraction of entities that can provide and accept services through well-defined named interfaces called ports. The CCM has a component container, where software components can be deployed. The container offers a set of services that the components can use. These services include (but are not limited to) notification, authentication, persistence and transaction processing. These are the most-used services any distributed system requires, and, by moving the implementation of these services from the software components to the component container, the complexity of the components is dramatically reduced. Portable interceptors are the "hooks", used by CORBA and RMI-IIOP to mediate the most important functions of the CORBA system. The CORBA standard defines the following types of interceptors:

- IOR interceptors mediate the creation of the new references to the remote objects, presented by the current server.
- Client interceptors usually mediate the remote method calls on the client (caller) side. If the object Servant exists on the same server where the method is invoked, they also mediate the local calls.
- Server interceptors mediate the handling of the remote method calls on the server (handler) side. The



interceptors can attach the specific information to the messages being sent and IORs being created. This information can be later read by the corresponding interceptor on the remote side. Interceptors can also throw forwarding exceptions, redirecting requests to another target.

General InterORB Protocol (GIOP)

Main article: General Inter-ORB Protocol The GIOP is an abstract protocol by which Object request brokers (ORBs) communicate. Standards associated with the protocol are maintained by the Object Management Group (OMG). The GIOP architecture provides several concrete protocols, including:

- 1. Internet InterORB Protocol (IIOP)** – The Internet Inter-Orb Protocol is an implementation of the GIOP for use over the Internet, and provides a mapping between GIOP messages and the TCP/IP layer.
- 2. SSL InterORB Protocol (SSLIOP)** – SSLIOP is IIOP over SSL, providing encryption and authentication.
- 3. HyperText InterORB Protocol (HTIOP)** – HTIOP is IIOP over HTTP, providing transparent proxy bypassing.
- 4. Zipped IOP (ZIOP)** – A zipped version of GIOP that reduces the bandwidth usage.

While CORBA delivered much in the way code was written and software constructed, it has been the subject of criticism. Much of the criticism of CORBA stems from poor implementations of the standard and not deficiencies of the standard itself. Some of the failures of the standard itself were due to the process by which the CORBA specification was created and the compromises inherent in the politics and business of writing a common standard sourced by many competing implementers.

Initial implementation incompatibilities

The initial specifications of CORBA defined only the IDL, not the on-the-wire format. This meant that source code compatibility was the best that was available for several years. With CORBA 2 and later this issue was resolved.

Location transparency

CORBA's notion of location transparency has been criticized; that is, that objects residing in the same address space and accessible with a simple function call are treated the same as objects residing elsewhere (different processes on the same machine, or different machines). This is a fundamental design flaw as it makes all object access as complex as the most complex case (i.e., remote network call with a wide class of failures that are not possible in local calls). It also hides the inescapable differences between the two classes, making it impossible for applications to select an appropriate use strategy (that is, a call with 1μs latency and guaranteed return will be used very differently from a call with 1s latency with possible transport failure, in which the delivery status is potentially unknown and might take 30s to time out).

Design and process deficiencies

The creation of the CORBA standard is also often cited for its process of design by committee. There was no process to arbitrate between conflicting proposals or to decide on the hierarchy of problems to tackle. Thus the standard was created by taking a union of the features in all proposals with no regard to their coherence. This made the specification complex, expensive to implement entirely, and often ambiguous. A design committee composed of a mixture of implementation vendors and customers created a diverse stem of

interests. This diversity made a cohesive standard difficult. Standards and interoperability increased competition and eased customers' movement between alternative implementations. This led to much political fighting within the committee and frequent releases of revisions of the CORBA standard that some ORB implementers ensured were difficult to use without proprietary extensions. Less ethical



CORBA vendors encouraged customer lock-in and achieved strong short-term results. Over time the ORB vendors that encourage portability took over market share.

Problems with implementations

Through its history, CORBA has been plagued by shortcomings in poor ORB implementations. Unfortunately, many of the papers criticizing CORBA as a standard are simply criticisms of a particularly bad CORBA ORB implementation.

CORBA is a comprehensive standard with many features. Few implementations attempt to implement all of the specifications, and initial implementations were incomplete or inadequate. As there were no requirements to provide a reference implementation, members were free to propose features which were never tested for usefulness or implementability.

Implementations were further hindered by the general tendency of the standard to be verbose, and the common practice of compromising by adopting the sum of all submitted proposals, which often created APIs that were incoherent and difficult to use, even if the individual proposals were perfectly reasonable. Robust implementations of CORBA have been very difficult to acquire in the past, but are now much easier to find. The SUN Java SDK comes with CORBA built-in. Some poorly designed implementations have been found to be complex, slow, incompatible and incomplete. Robust commercial versions began to appear but for significant cost. As good quality free implementations became available the bad commercial implementations died quickly.

Firewalls

CORBA (more precisely, GIOP) is not tied to any particular communications transport. A specialization of GIOP is the Internet Inter-ORB Protocol or IIOP. IIOP uses raw TCP/IP connections in order to transmit data. If the client is behind a very restrictive firewall or transparent proxy server environment that only allows HTTP connections to the outside through port 80, communication may be impossible, unless the proxy server in question allows the HTTP CONNECT method or SOCKS connections as well. At one time, it was difficult even to force implementations to use a single standard port – they tended to pick multiple random ports instead. As of today, current ORBs do have these deficiencies. Due to such difficulties, some users have made increasing use of web services instead of CORBA. These communicate using XML/SOAP via port 80, which is normally left open or filtered through a HTTP proxy inside the organization, for web browsing via HTTP. Recent CORBA implementations, though, support SSL and can be easily configured to work on a single port. Some ORBs, such as TAO, omniORB and JacORB also support bidirectional GIOP, which gives CORBA the advantage of being able to use callback communication rather than the polling approach characteristic of web service implementations. Also, most modern firewalls support GIOP & IIOP and are thus CORBA friendly firewalls.

Benefit:

CORBA's benefits include language- and OS-independence, freedom from technology-linked implementations, strong data-typing, high level of tenability, and freedom from the details of distributed data transfers.

Language independence

CORBA was designed to free engineers from limitations of coupling their designs to a particular software language. Currently there are many languages supported by various CORBA providers, the most popular being Java and C++. There are also C++11, C-only, Smalltalk, Perl, Ada, Ruby, and Python



implementations, just to mention a few.

OS-independence

CORBA's design is meant to be OS-independent. CORBA is available in Java (OS-independent), as well as natively for Linux/Unix, Windows, Solaris, OS X, OpenVMS, HP/UX, Android, LynxOS, VxWorks, ThreadX, INTEGRITY, and others.

Freedom from technologies

One of the main implicit benefits is that CORBA provides a neutral playing field for engineers to be able to normalize the interfaces between various new and legacy systems. When integrating C, C++, Object Pascal, Java, Fortran, Python, and any other language or OS into a single cohesive system design model, CORBA provides the means to level the field and allow disparate teams to develop systems and unit tests that can later be joined together into a whole system. This does not rule out the need for basic system engineering decisions, such as threading, timing, object lifetime, etc. These issues are part of any system regardless of technology. CORBA allows system elements to be normalized into a single cohesive system model.

For example, the design of a multitier architecture is made simple using Java Servlets in the web server and various CORBA servers containing the business logic and wrapping the database accesses. This allows the implementations of the business logic to change, while the interface changes would need to be handled as in any other technology. For example, a database wrapped by a server can have its database schema changed for the sake of improved disk usage or performance (or even whole-scale database vendor change), without affecting the external interfaces. At the same time, C++ legacy code can talk to C/Fortran legacy code and Java database code, and can provide data to a web interface.

Data-typing

CORBA provides flexible data typing, for example an "ANY" datatype. CORBA also enforces tightly coupled data typing, reducing human errors. In a situation where Name-Value pairs are passed around, it is conceivable that a server provides a number where a string was expected. CORBA Interface Definition Language provides the mechanism to ensure that user-code conforms to method-names, return-, parameter-types, and exceptions.

High tunability

Many implementations (e.g. ORBexpress (Ada, C++, and Java implementation) and OmniORB (open source C++ and Python implementation)) have options for tuning the threading and connection management features. Not all ORB implementations provide the same features.

Freedom from data-transfer details

When handling low-level connection and threading, CORBA provides a high level of detail in error conditions. This is defined in the CORBA-defined standard exception set and the implementation-specific extended exception set. Through the exceptions, the application can determine if a call failed for reasons such as "Small problem, so try again", "The server is dead" or "The reference does not make sense." The general rule is: Not receiving an exception means that the method calls completed successfully. This is a very powerful design feature.

Compression

CORBA marshals its data in a binary form and supports compression. IONA, Remedy IT, and Telefónica have worked on an extension to the CORBA standard that delivers compression. This extension is called ZIOP and this is now a formal OMG standard.

Experiment No. 10

Aim: Case Study On Android Stack.

OVERVIEW:

Android Architecture android architecture or Android software stack is categorized into five parts: 1. linux kernel

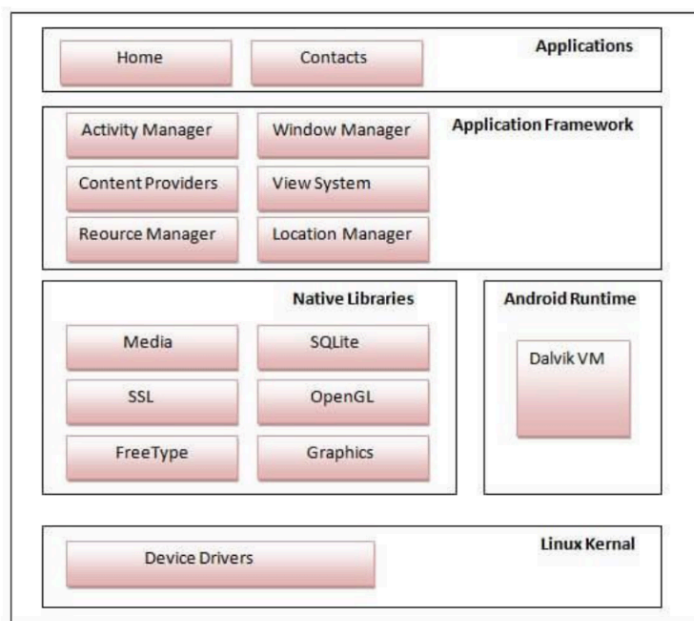
2. native libraries (middleware),

3. Android Runtime

4. Application Framework

5. Applications

Let's see the android architecture first.



1) Linux kernel

It is the heart of android architecture that exists at the root of android architecture. Linux kernel is responsible for device drivers, power management, memory management, device management and resource access.

2) Native Libraries

On the top of the Linux kernel, there are Native libraries such as WebKit, OpenGL, FreeType, SQLite, Media, C runtime library (libc) etc. The WebKit library is responsible for browser support, SQLite is for database, Free Type for font support, Media for playing and recording audio and video formats.

3) Android Runtime

In android runtime, there are core libraries and DVM (Dalvik Virtual Machine) which is responsible for running android applications. DVM is like JVM but it is optimized for mobile devices. It consumes less memory and provides fast performance.

4) Android Framework

On the top of Native libraries and android runtime, there is an android framework. Android framework includes Android APIs such as UI (User Interface), telephony, resources, locations, Content Providers (data) and package managers. It provides a lot of classes and interfaces for android application



development.

5) Applications

On the top of the android framework, there are applications. All applications such as home, contact, settings, games, browsers are using android framework that uses android runtime and libraries. Android runtime and native libraries are using the linux kernel.

Linux Kernel

Android was created on the open-source kernel of Linux. One main reason for choosing this kernel was that it provided proven core features on which to develop the Android operating system. The features of the Linux kernel are:

1. Security: The Linux kernel handles the security between the application and the system. **2.**

Memory Management: It efficiently handles memory management thereby providing the freedom to develop our apps.

3. Process Management: It manages the process well and allocates resources to processes whenever they need them.

4. Network Stack: It effectively handles network communication.

5. Driver Model: It ensures that the application works. Hardware manufacturers can build their drivers into the Linux build.

Libraries:

Running on top of the kernel, the Android framework was developed with various features. It consists of various C/C++ core libraries with numerous open-source tools. Some of these are:

1. The Android Runtime: The Android runtime consists of core libraries of Java and ART(the Android RunTime). Older versions of Android (4.x and earlier) had Dalvik runtime.

2. Open GL(graphics library): This cross-language, cross-platform application program interface (API) is used to produce 2D and 3D computer graphics.

3. WebKit: This open-source web browser engine provides all the functionality to display web content and simplify page loading.

4. Media frameworks: These libraries allow you to play and record audio and video. **5. Secure Socket**

Layer (SSL): These libraries are there for Internet security. Android Runtime It is the third section of the architecture. It provides one of the key components which is called Dalvik Virtual Machine. It acts like Java Virtual Machine which is designed especially for Android. Android uses its own custom VM designed to ensure that multiple instances run efficiently on a single device. The Dalvik VM uses the device's underlying Linux kernel to handle low-level functionality, including security, threading, and memory management.

Application Framework

The Android team has built on a known set of proven libraries, built in the background, and all of it is exposed through Android interfaces. These interfaces wrap up all the various libraries and make them useful for the Developer. They don't have to build any of the functionality provided by the android. Some of these interfaces include:

1. Activity Manager: It manages the activity lifecycle and the activity stack.

2. Telephony Manager: It provides access to telephony services as related subscriber information, such as phone numbers.

3. View System: It builds the user interface by handling the views and layouts.

4. Location manager: It finds the device's geographic location.



Applications:

Android applications can be found at the topmost layer. At the application layer, we write our application to be installed on this layer only. Examples of applications are Games, Messages, Contacts, etc. There are some necessary building blocks that an Android application consists of. These loosely coupled components are bound by the application manifest file which contains the description of each component and how they interact. The manifest file also contains the app's metadata, its hardware configuration, and platform requirements, external libraries, and required permissions. There are the following main components of an android app:

1. Activities

Activities are said to be the presentation layer of our applications. The UI of our application is built around one or more extensions of the Activity class. By using Fragments and Views, activities set the layout and display the output and also respond to the user's actions. An activity is implemented as a subclass of class Activity.

- Java
- Kotlin

```
public class MainActivity extends Activity {  
  
}
```

2. Services

Services are like invisible workers of our app. These components run at the backend, updating your data sources and Activities, triggering Notification, and also broadcast Intents. They also perform some tasks when applications are not active. A service can be used as a subclass of class Service:

- Java
- Kotlin

```
public class ServiceName extends Service {  
  
}
```


3. Content Providers

It is used to manage and persist the application data and also typically interacts with the SQL database. They are also responsible for sharing the data beyond the application boundaries. The Content Providers of a particular application can be configured to allow access from other applications, and the Content Providers exposed by other applications can also be configured.

A content provider should be a subclass of the class Content Provider

- | |
|----------|
| ● Java |
| ● Kotlin |

```
import androidx.appcompat.app.AppCompatActivity;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;

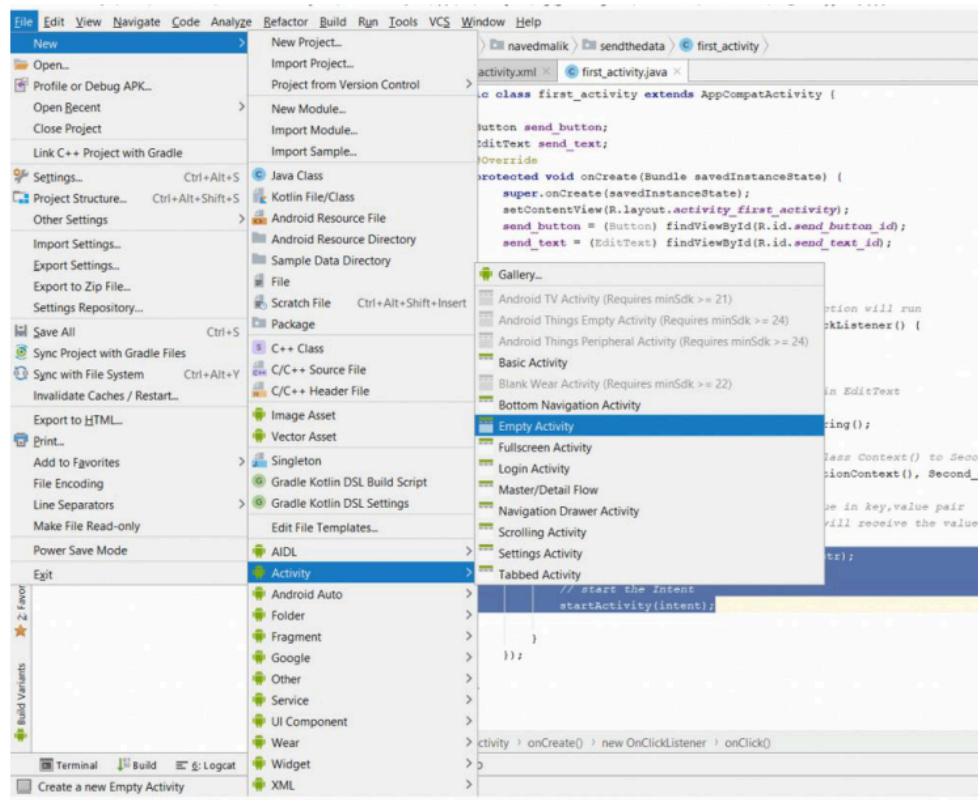
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    public void newsScreen(View view) {
        Intent i = new Intent(getApplicationContext(), MainActivity2.class);
        startActivity(i);
    }
}
```

4. Working with the activity_main2.xml File

Now we have to create a second activity as a destination activity. The steps to create the second activity are File > new > Activity > Empty Activity.



Next, go to the activity_main2.xml file, which represents the UI of the project. Below is the code for the activity_main2.xml file. Comments are added inside the code to understand the code in more detail.

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<androidx.constraintlayout.widget.ConstraintLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    xmlns:app="http://schemas.android.com/apk/res-auto"
```

```
    xmlns:tools="http://schemas.android.com/tools"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    tools:context=".MainActivity2">
```

```
<TextView
```

```
    android:id="@+id/editText"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Welcome to GFG News Screen"
```

```
    android:textAlignment="center"
```

```
    android:textSize="28sp"
```

```
    app:layout_constraintBottom_toBottomOf="parent"
```

```
    app:layout_constraintEnd_toEndOf="parent"
```

```
    app:layout_constraintHorizontal_bias="0.0"
```

```
    app:layout_constraintStart_toStartOf="parent"
```

```
    app:layout_constraintTop_toTopOf="parent" />
```

<Button

```
android:id="@+id/btn2"
android:text="Go to Home Screen"
android:onClick="homeScreen"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
app:layout_constraintBottom_toBottomOf="parent"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toBottomOf="@+id/editText" />
```

</androidx.constraintlayout.widget.ConstraintLayout>

5. Working with the MainActivity2 File

Now, we will create the Backend of the App. For this, Open the MainActivity file and instantiate the component (Button, TextView) created in the XML file using the findViewById() method. This method binds the created object to the UI Components with the help of the assigned ID.

Syntax:

```
ComponentType object = (ComponentType) findViewById(R.id.IdOfTheComponent);
Intent i = new Intent(getApplicationContext(), <className> );
startActivity(i);
```

```
import androidx.appcompat.app.AppCompatActivity;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
```

```
public class MainActivity2 extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main2);
    }

    public void homeScreen(View view) {
        Intent i = new Intent(getApplicationContext(), MainActivity.class);
        startActivity(i);
    }
}
```